2026

IIT ROORKEE MOTORSPORTS × dsg

AUTONOMOUS

Recruitment Manual

IIT Roorkee Motorsports

x

Data Science Group

Contents

A. Perception **4**
1. Understanding Images 4
2. CNNs 4
3. Object Detection 6
4. Object Segmentation 6
7
5. Object Tracking 7
6. LIDAR (Light Detection and Ranging) sensor 8
7. Stereo Camera 8
B. SLAM **11**
1. Introduction to SLAM 11
2. Core Components of SLAM 12
I. Range-Bearing Measurement 12
II. Data Processing and Extraction 12
III. Data Association and Mapping 12
IV. Loop Closure 12
3. SLAM Techniques for Motorsports based Navigation 13
I. LIDAR-Only SLAM 13
II. General Architecture of LIDAR + Wheel Odometry 13
III. VISUAL SLAM 14
IV. EKF-SLAM 15
4. Deep Learning in SLAM 15
C. Path Planning **16**
1. What is Path Planning? 16
2. Importance of Path Planning in Autonomous Vehicles 16
3. Overview of Path Planning Algorithms 16
I. Artificial Potential Field (APF) 17
II. Rapidly-Exploring Random Tree 17

III. A* and D* Algorithms 17
IV. Delaunay Triangulation and Voronoi Diagrams 18
V. Deep Learning Methods 18
D. Control 19
1. Introduction 19
2. Open Loop v/s Closed Loop 20
3. PID : 20
E. Vehicle Dynamics 22
1. Load Transfer 22
2. Coordinates 22
3. Steering 23
I. Role of Steering System 23
II. Types of Steering Systems 23
4. Braking 24
Key Concepts in Braking Calculations 25
F. Powertrain 26
1. Torque-RPM Curve 26
I. Initial Region (Constant Torque) 26
II. Transition to Maximum Power (Base Speed) 26
III. Constant Power Region (Hyperbolic Torque Decrease) 26
2. Powertrain Configurations 27
I. Single Motor Configuration 27
II. Dual Motor Configuration 28
III. Four Motor Configuration 28
3. Motor and its Types 28
I. Motor Control Unit 29
G. Robot Operating System (ROS) 30
1. ROS and its use in Autonomous Vehicles: 30
2. Use of Version Control in ROS: 32
3. Linux, Virtual Machines & Programming Languages: 32

A. Perception

1. Understanding Images

In computer vision, an image can be either grayscale or color, depending on whether it contains information about light intensity alone or the combination of different colors. The images are composed of pixels, where each pixel has an intensity value. These images are stored as matrices, where:

- A grayscale image consists of a single channel of intensity values ranging from black to white. This means it is represented as a 2D matrix with pixel values from 0(black) to 255(white).
- A color image has multiple channels, each representing a different color channel such as red, green, and blue (RGB). They are represented by three 2D matrices, one for each color channel. Their intensity values also vary from 0 to 255.

Some image characteristics are:

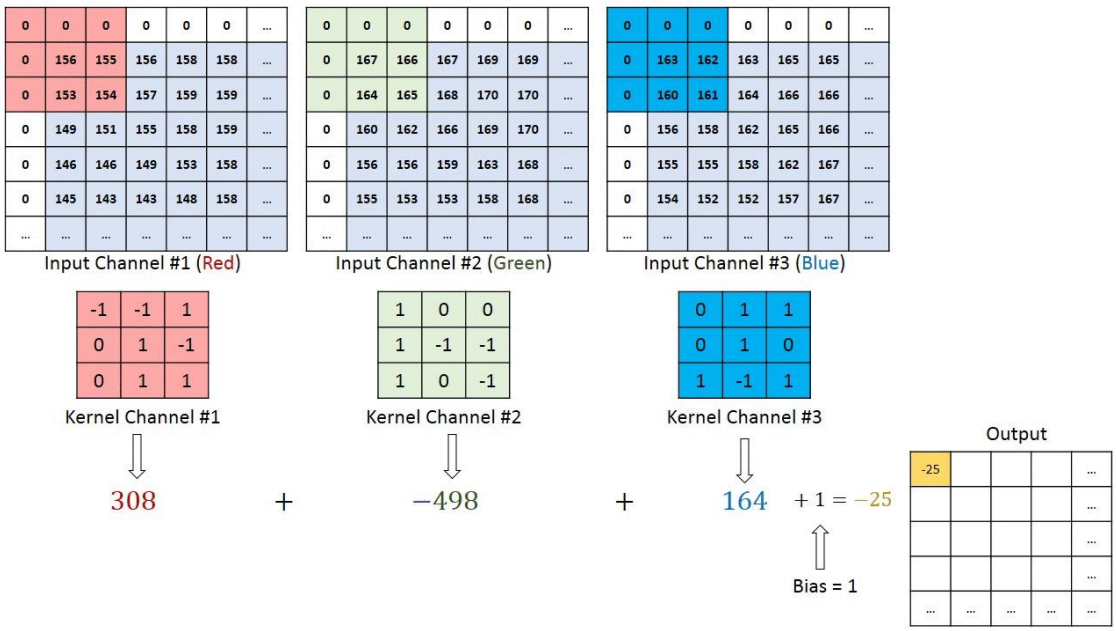
- **Resolution:** The resolution of an image refers to the number of pixels it contains. A higher resolution image will contain more pixels and thus more detail.
- **Bit Depth:** Bit depth refers to the number of bits used to represent each pixel in an image. A higher bit depth allows for more color and brightness variations in an image (since the number of possible intensity values that can be represented increase).
- **Noise:** Noise refers to any unwanted artifacts or variations in an image. It can be caused by factors such as sensor sensitivity, lighting conditions, or image compression.

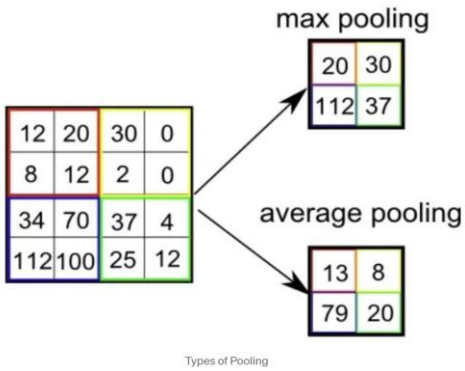
For computers, understanding images requires mathematical operations on these matrices. These mathematical operations help the computer in detecting edges, textures, corners etc. This is where Convolutional Neural Networks (CNNs) come into play.

2. CNNs

As seen below, CNNs are neural networks where we have convolutional layers instead of fully connected layers. Convolutional layers consist of “kernels,” which are matrices. The output of each layer is another matrix known as feature maps, whose entries are calculated as the sum of input entries weighted by kernel entries. The kernel sweeps over the entire input matrix (the image is represented as a matrix). In the case of multiple channels, the weighted sums of each channel with its kernel are added to give output. The convolutional layers learn features of images from simple things such as edges or some shapes to more complicated things like facial features or complete objects.

CNNs also consist of pooling layers and fully connected layers. Pooling layers are used to reduce the dimensionality of feature maps. Fully connected layers are generally added at the network's end to predict class probabilities in object identification tasks.





Most vision architectures use convolutions as a fundamental building block.

3. Object Detection

Object detection involves identifying and localizing objects in an image. For localization, bounding boxes are drawn around the identified object. Object detection algorithms employ a combination of techniques to accomplish their tasks effectively. They utilize Region Proposal Networks (RPNs) to generate potential object proposals. These proposals are then analyzed by feature extraction networks, which extract relevant features to accurately classify and localize objects within the image or video.

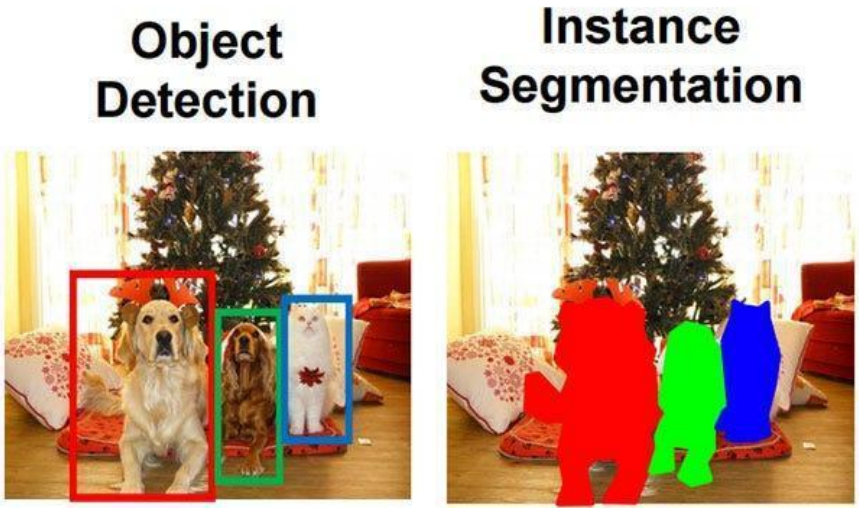
Some popular object detection models include:

- **Faster R-CNN** (Region-based Convolutional Neural Network) : Uses a Region Proposal Network (RPN) to efficiently detect objects with high accuracy.
- **YOLO** (You Only Look Once) : A fast and real-time object detection model that processes an image in a single pass.

4. Object Segmentation

Object Segmentation or image segmentation refers to categorizing each pixel value of an image to a particular class. This results in a pixel-wise mask for the objects. Types include:

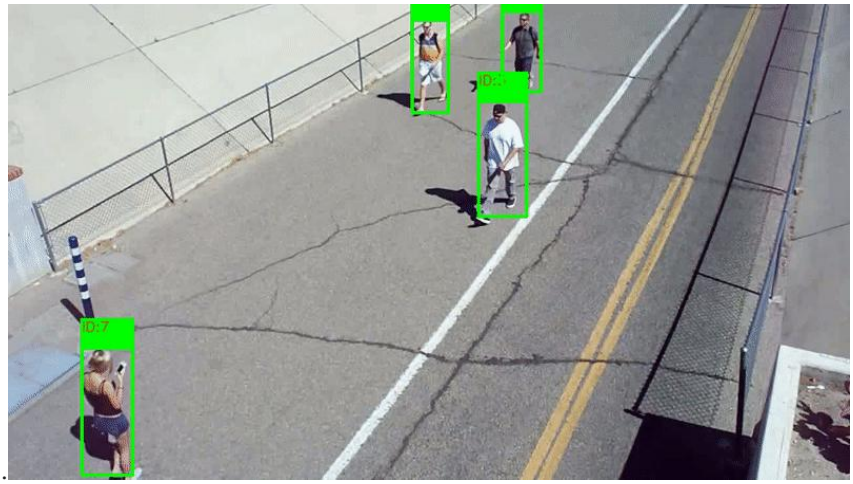
- **Semantic Segmentation:** Classifies pixels into categories (e.g., road, car, pedestrian).
- **Instance Segmentation:** Differentiates between multiple instances of the same object (e.g., two different cars).



5. Object Tracking

Object tracking is the process of locating and following an object's movement in a video or a sequence of frames. Object tracking aims to identify an object of interest in the first frame and track it as it moves in subsequent frames. The object tracking process is terminated when the object of interest is no longer visible or the tracking algorithm fails to track the object accurately.

It is crucial for autonomous driving to track objects that need to be avoided (like other cars, pedestrians, or any other obstacles) or followed(lane boundaries need to be tracked to ensure the vehicle stays in the lane)



Some common problems in object racking include:

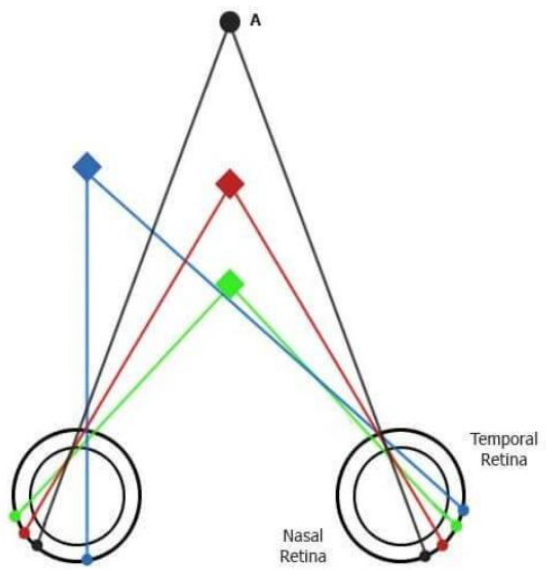
- **Occlusion:** It is known as occlusion when the object is hidden partially or entirely due to other objects. Solutions include using multiple detectors to detect different parts or a Kalman filter to predict the expected position of the object based on its motion in the previous frame.
- **Motion blur:** Sometimes, objects can appear blurred in video frames due to fast motion. To handle this Kalman filter can be used to track using the expected position until a sharp object is found again.

6. LIDAR (Light Detection and Ranging) sensor

Cameras provide 2D images of the environment, but autonomous cars need 3D information like the distance of objects from the vehicle or the velocity of other objects to make decisions, such as when to turn or stop. LIDAR sensor is used for this purpose:

- Emits laser pulses and measures the reflected time to compute distance (like sonar, except it uses light pulses instead of sound waves, which are significantly slower).
- Generates a 3D point cloud representing the surroundings.
- It helps in depth and velocity estimation, which is critical for navigation.

7. Stereo Camera



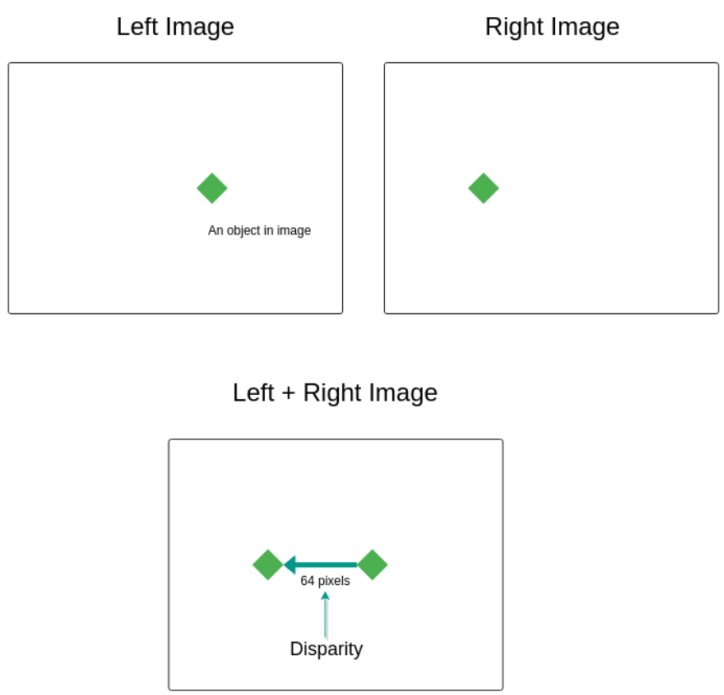
A stereo camera is a type of camera with two or more image sensors. This allows the camera to simulate human binocular vision and therefore gives it the ability to perceive depth.

The human binocular vision perceives depth by using Stereo disparity which refers to the difference in image location of an object seen by the left and right eyes, resulting from the eyes' horizontal separation.

The brain uses this binocular disparity to extract depth information from the two-dimensional retinal images which are known as stereopsis.

Stereo Disparity in a camera can be found by using two 2D images taken from different positions and the correlation between the images can be used to create a depth image.

However, to find correlations the two images need to have sufficient details and texture or non-uniformity.



Thus, the Stereo vision is suitable for applications with a large field of view and for outdoor usage.

1. ORB (Oriented FAST and Rotated BRIEF)

ORB is a fast, efficient algorithm designed to detect "keypoints" (interesting features) in an image and describe them so they can be matched across different frames. It was created as a computationally efficient, open-source alternative to SIFT and SURF.

ORB is actually a fusion of two other algorithms, modified to handle rotation:

1. **Feature Detector:** Oriented FAST (oFAST)
2. **Feature Descriptor:** Rotated BRIEF (rBRIEF)

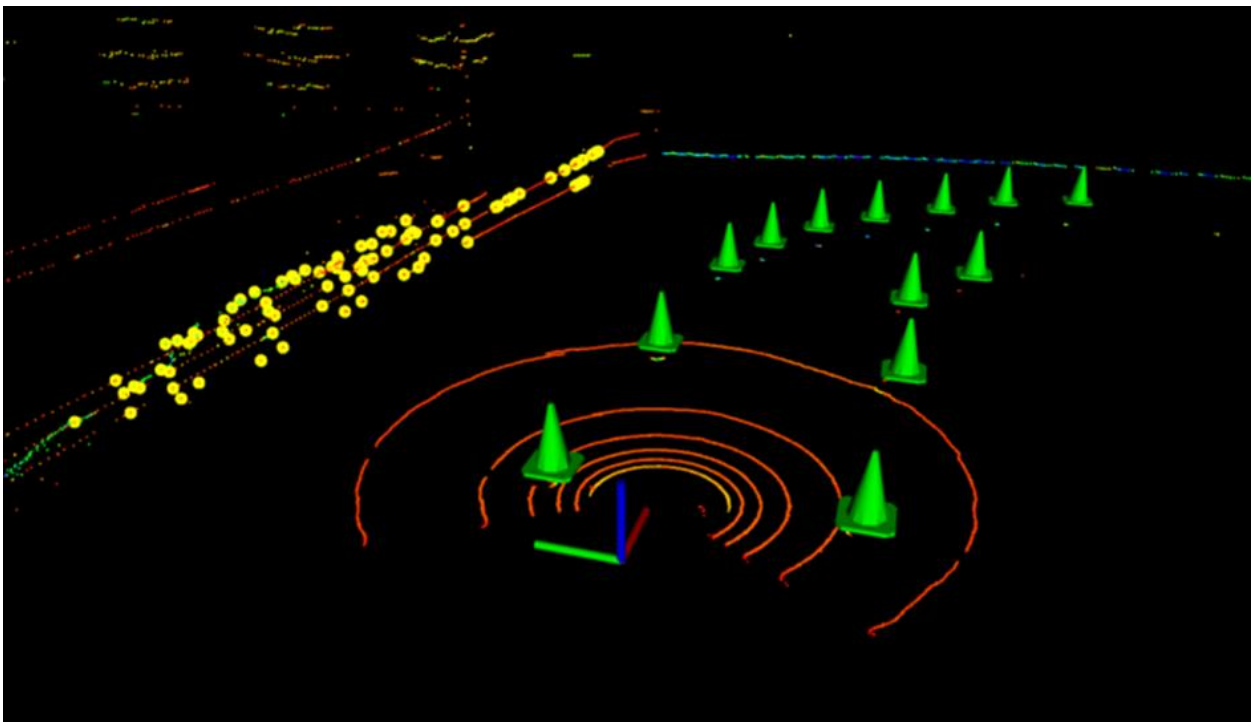
Links to referred resources:

- https://medium.com/towards-data-science/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53
- https://keymakr.com/blog/semantic-segmentation-vs-object-detection-understanding-the-differences/#:~:text=Segmentation%20focuses%20on%20dividing%20an,within%20an%20image%20or%20video.
- LIDAR Sensor in Autonomous Vehicles its Importance for Self-Driving Cars
- https://medium.com/@ajithraj_gangadharan/3d-ransac-algorithm-for-lidar-pcd-segmentation-315d2a51351
- https://youtu.be/RDZUdRSDOok?si=QfphM9HxWJl9Xhc-
- https://www.basic.ai/blog-post/camera-lidar-sensor-fusion-key-concept-intrinsic-parameters
- https://www.e-consystems.com/blog/camera/technology/what-is-a-stereo-vision-camera-2/
- https://ieeexplore.ieee.org/document/6126544
- https://www.geeksforgeeks.org/python/feature-matching-using-orb-algorithm-in-python-opencv/

- https://ieeexplore.ieee.org/document/6126544
- https://www.geeksforgeeks.org/python/feature-matching-using-orb-algorithm-in-python-opencv/

B. SLAM

1. Introduction to SLAM



https://www.mathworks.com/discovery/slam.html
https://www.mdpi.com/2072-4292/15/4/1156 (Intensive one)

Simultaneous Localization and Mapping (SLAM) is a groundbreaking technology that allows autonomous systems, such as robots or vehicles, to navigate and understand unfamiliar environments. It achieves this by simultaneously creating a map of the surroundings and determining the system's position within that map. This dual challenge was once considered a "chicken-or-egg" problem in robotics, as mapping requires knowing the location, and localization requires a map. SLAM elegantly solves this interdependency.

In essence, SLAM enables machines to explore unknown spaces much like humans do when navigating new places by identifying landmarks and using them as reference points.

For example:

- A robotic vacuum cleaner uses SLAM to map your house while cleaning, ensuring it doesn't miss spots or repeatedly clean the same area.
- A self-driving car uses SLAM to navigate roads safely by continuously updating its map of the surroundings.

2. Core Components of SLAM

SLAM systems are built on two essential pillars:

I. Range-Bearing Measurement & Prediction

Every SLAM system relies on a sensing mechanism that enables a robot or vehicle to perceive and measure its surroundings. This is achieved through various technologies, including cameras, image sensors, LiDAR (Light Detection and Ranging), and sonar. Essentially, any device capable of detecting spatial properties such as location, distance, and velocity can serve as a range measurement tool within a SLAM framework.

II. Data Processing and Extraction

Once environmental measurements are collected, the SLAM system employs computational methods to process and interpret this data. The type of data ranges from the accelerometer readings from the car's body to the positional observance of objects surrounding the vehicle. The primary objective of this data extraction process is to obtain the necessary data to help aid the calculations of position, acceleration, velocity and yaw.

A well-integrated SLAM system continuously synchronizes its sensing hardware, data processing algorithms, and the movement of the robot or vehicle itself. This interplay ensures that the system can accurately explore and navigate its surroundings. While the specific components and technologies used may vary depending on the application, their seamless coordination is essential for the system's effectiveness.

III. Data Association and Mapping

Reference: https://web.mit.edu/2.166/www/handouts/2166_data_association_jose_neira.pdf

Data Association is one of the key features governing a SLAM algorithm. To accurately map and recognise its own position within the virtual world, the algorithm must relate each new observed frame with the preceding one based on estimates to determine where the vehicle should be after the duration of time between the observed frames. Many such algorithms even accommodate the use of gating regions, in an attempt to simplify the association calculations.

IV. Loop Closure

Example- https://www.youtube.com/watch?v=lThcTKYE7Es&t=147s

Loop closure is a critical feature of SLAM that ensures accuracy over time. It involves recognizing previously visited areas and correcting any drift in localization or mapping errors.

For example: Imagine a robot exploring a building. When it revisits a hallway it mapped earlier, loop closure ensures that any discrepancies between its current position and the earlier map are corrected.

Pose Graph Optimization:
https://www.youtube.com/watch?v=saVZtgPyyJQ&list=PLn8PRpmsu08rLRGrnF-S6TyGrmcA2X7kg&index=3

3. SLAM Techniques for Motorsports based Navigation

I. LIDAR-Only SLAM

LiDAR (Light Detection and Ranging) is a popular choice for SLAM in motorsports due to its ability to provide precise 3D point cloud data. This data enables the detection and localization of objects like traffic cones, which are essential for defining the racetrack.

Steps in LiDAR SLAM for Cone Detection

- Preprocessing Point Cloud Data:
  - Field of View Trimming: Limits the sensor's range to focus only on relevant areas.
  - Ground Plane Segmentation: Separates the ground from objects like cones to simplify detection.
- Clustering Potential Obstacles:
  - Groups point in the cloud that likely belong to the same object (e.g., a cone).
- Key Techniques:
  - Cylindrical Volume Reconstruction: Ensures all points within a cylindrical region around each cluster are included.
  - Filtering Based on Expected Point Counts: Verifies if the number of points in a cluster matches the expected size of a cone.
- Scan Matching Algorithms:
  - These algorithms align consecutive LiDAR scans to maintain accurate localization:
  - - Iterative Closest Point (ICP): Matches corresponding points between scans to minimize positional errors.

Videos - https://www.youtube.com/watch?v=4uWSo8v3iQA

  - - Normal Distribution Transform (NDT): Models point clouds as Gaussian distributions, making them robust against noise.

Blog - https://medium.com/towards-data-science/types-of-transformations-for-better-normal-distribution-61c22668d3b9

II. General Architecture of LIDAR + Wheel Odometry

For motorsports navigation, combining LiDAR with wheel odometry enhances accuracy. Here's how it works:

- Wheel odometry provides initial estimates of the vehicle's movement based on wheel rotations.
- LiDAR refines these estimates by comparing real-world measurements with the map being built.

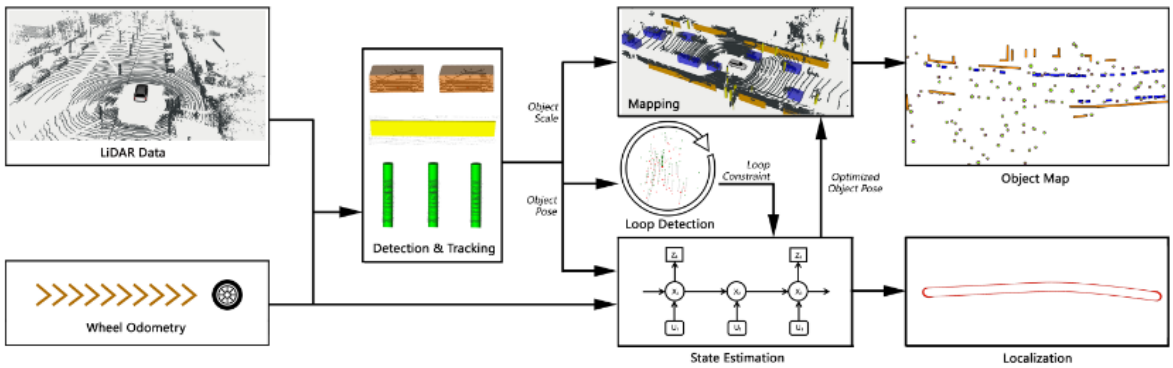


Fig. 2. The pipeline of the proposed object-oriented SLAM system. A detection and tracking module finds vehicles, walls, and poles from the LiDAR data. The measurements of the objects and wheel odometry are integrated to build a factor graph for pose estimation. A loop detection module searches for loops from the detection, and a mapping module produces an object-based map of the environment.

This fusion ensures precise localization even at high speeds or during sharp turns.

reference - https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9636299)

III. VISUAL SLAM

Visual SLAM uses cameras instead of LiDAR to detect and map objects. It is often combined with LiDAR for enhanced performance in motorsports.

Feature-Based Methods:

- Extracts distinct features (e.g., corners or edges) from images.
- Tracks these features across frames to estimate motion and reconstruct the environment.
- Example: ORB-SLAM, which uses ORB (Oriented FAST and Rotated BRIEF) features for real-time mapping.(examples - https://www.youtube.com/watch?v=djZKOmNs7sA&list=PL1nA2S3vm3wBQFuLKzq1FdLLljn4NaGEf)

Direct Methods:

- Uses all pixel intensities instead of specific features.
- Estimates motion by comparing brightness values between consecutive frames.
- Works well in environments with minimal texture but is sensitive to lighting changes.

Advantages:

- Cost-effective compared to LiDAR.
- Provides rich visual information, such as cone color or texture.

Challenges:

- Less reliable in low-light or high-speed scenarios without additional processing.
- Computationally demanding when producing dense maps.

IV. EKF-SLAM

EKF-SLAM estimates both the robot's position and the locations of landmarks (e.g., cones) in the environment. It alternates between:

- **Prediction**: Uses the robot's motion model to predict its next position, increasing uncertainty due to process noise.
- **Correction**: Updates the prediction using observed landmarks, reducing uncertainty by comparing predicted and actual sensor measurements.

For example, a robot navigating a cone-marked track predicts its movement using odometry and refines its position when it observes a cone landmark.

Variations of EKF-SLAM for Non-Linear Trends : First Estimates Jacobian EKF (FEJ-EKF), Iterated EKF (IEKF), Unscented Kalman Filter (UKF), Submap-Based EKF-SLAM, Dynamic SLAM Variants.

Other such variations include a graph based SLAM or a Particle Filter Based SLAM involving algorithms like Monte Carlo Simulations or a field based recursion.

You can refer to this article for more concreteness in EKF-SLAM -
https://jihongju.github.io/2018/10/05/slam-05-ekf-slam/

Do check out this video to understand how randomness exists via a Monte Carlo Simulation to build intuition:
https://www.youtube.com/watch?v=7ESK5SaP-bc

And also explore this approach if interested in diving deeper into how SLAM utilises it:

https://robots.stanford.edu/papers/Thrun03g.pdf

4. Deep Learning in SLAM

Deep Learning (DL) has emerged as a transformative tool in Simultaneous Localization and Mapping (SLAM), offering data-driven solutions to challenges that traditional geometry-based approaches struggle with. By leveraging neural networks, SLAM systems can achieve greater robustness, adaptability, and efficiency, especially in dynamic or complex environments. Below, we scratch the surface of this exciting intersection by exploring key architectures, challenges, and their potential fixes.

Reinforcement learning can also see scope among the many existing algorithms for high speed localisation and mapping. Parameter tuning, Multiple Map Dynamics or even Per frame recognition can be done on a learning model over huge data.

You are encouraged to explore more ideas of how RL can help improve costs or accuracy of certain steps in existing SLAM pipelines like what's discussed in the reference below.

Reference: https://arxiv.org/pdf/2408.14518

C. Path Planning

1. What is Path Planning?

Path planning is a fundamental component of autonomous vehicle systems that determines a safe, efficient, and feasible trajectory for a vehicle from its starting position to a desired destination. The primary goal is to navigate through a dynamic environment while avoiding obstacles, adhering to traffic rules, and optimizing factors such as time, energy, and comfort. It is the process of determining the optimal route from a starting point to a destination while avoiding obstacles and adhering to rules and regulations.

2. Importance of Path Planning in Autonomous Vehicles

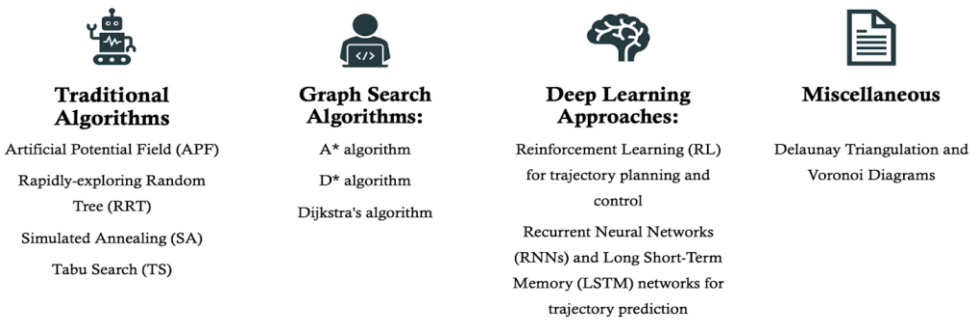
For autonomous vehicles, effective path planning is crucial because it directly impacts safety, efficiency, and overall performance. A well-planned path ensures:

- Safety: Avoids collisions with static and dynamic obstacles.
- Efficiency: Finds the shortest or most optimal route based on criteria like travel time, energy consumption, or passenger comfort.
- Adaptability: Responds to real-time changes in traffic, road conditions, or unpredictable obstacles.

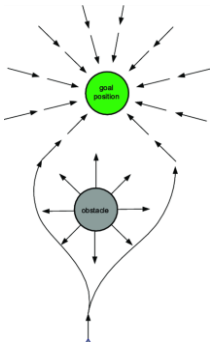
Path planning can be broadly categorized into two types based on the level of planning and how dynamic the environment is:

- **Global planning:** A high-level route between the source to destination based on the surroundings and track of the autonomous vehicle. It assumes a known, static environment and does not account for real-time changes such as moving obstacles or traffic conditions.
- **Local planning:** Handling real-time navigation and obstacle avoidance on the track. Unlike global planning, as the name suggests, it reacts to changes in the environment as the vehicle moves.

3. Overview of Path Planning Algorithms



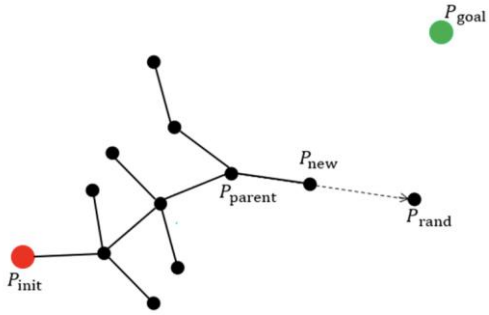
I. Artificial Potential Field (APF)



The Artificial Potential Field (APF) method is a local path planning technique widely used in autonomous vehicles and robotics for real-time obstacle avoidance. It models the environment using a mathematical field where the goal exerts an attractive force, and obstacles exert repulsive forces. The vehicle navigates by following the resultant force, ensuring a smooth and safe trajectory.

Reference: [Predictive Artificial Potential Field algorithm - energy-efficient local path planning algorithm]

II. Rapidly-Exploring Random Tree



This is a sampling-based method (in that it generates random paths to find the optimal path). It allows efficient exploration of high-dimensional spaces by incrementally building a tree structure.

It consists of three steps, viz. exploration, obstacle avoidance and path formation:

Reference: [RRT* Algorithm Explained]

III. A* and D* Algorithms

Both A* and D* are heuristic-based path planning algorithms widely used in robotics, AI, and autonomous vehicles. A* is used for static environments, while D* is an extension designed for dynamic, changing environments.

A* Algorithm

- A* is a heuristic-based graph-based search algorithm that finds the shortest path from a start node to a goal node. It combines concepts from Dijkstra's Algorithm (which finds the shortest path but explores too many nodes) and Greedy Best-First Search (which is fast but may not always find the best path)

D* (Dynamic A*) Algorithm

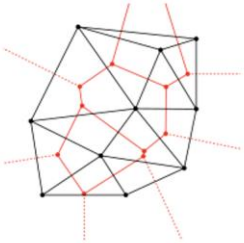
- D* is an extension of A* designed for dynamic environments where obstacles may change after path computation. It is widely used in robotics and autonomous vehicles to replan paths when new obstacles appear.

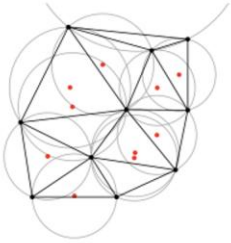
Reference: [D* Lite Path Planning Algorithm], [A* (A Star) Search Algorithm - Computerphile]

IV. Delaunay Triangulation and Voronoi Diagrams

Delaunay triangulation and Voronoi diagrams are geometric structures used in computational geometry and robotics. They help in path planning, obstacle avoidance, and spatial analysis in autonomous vehicles in simple track situations like those in FS AI.

Voronoi Diagram
A Voronoi diagram partitions a plane into regions based on proximity to a given set of points. Each region contains all points that are closer to one specific point than to any other.





Delaunay Triangulation
Delaunay triangulation is a way of connecting a set of points with non-overlapping triangles such that the circumcircle of each triangle contains no other points. It is essentially equivalent to KNN with k=1 (ML algorithm).

V. Deep Learning Methods

RNNs and LSTMs are some architectures, alongside RL algorithms that can be used for various parts of path planning. RL is increasingly used to replace the "Local Planner" or "Controller"—the component that decides how to steer *right now* to stay on the optimal path calculated by the global planner.

1. **Soft Actor-Critic (SAC) & TD3:**
    a. **Role:** Continuous control (steering angle and throttle/brake).
    b. **Application:** These off-policy algorithms are sample-efficient and stable. They are trained in simulators (like F1TENTH Gym or CARLA) to follow a racing line while minimizing lap time.
    c. **Advantage:** Unlike PID or pure pursuit, an RL agent can learn to "slip" the tires slightly or brake late, exploiting vehicle dynamics that are hard to model mathematically.
2. **Proximal Policy Optimization (PPO):**
    a. **Role:** Robust, on-policy control.
    b. **Application:** Often used for **obstacle avoidance** (e.g., in the FSUK skidpad or trackdrive if dynamic obstacles were introduced). It is easier to tune than SAC but typically requires more training data.

We look forward to you being enthusiastic and interested in these type of methods to innovate and think over the implications and usage of the current AI bubble to develop and research on these deep learning based methods.
Do check out this video:
https://www.youtube.com/playlist?list=PLn8PRpmsu08rLRGrnF-S6TyGrmcA2X7k

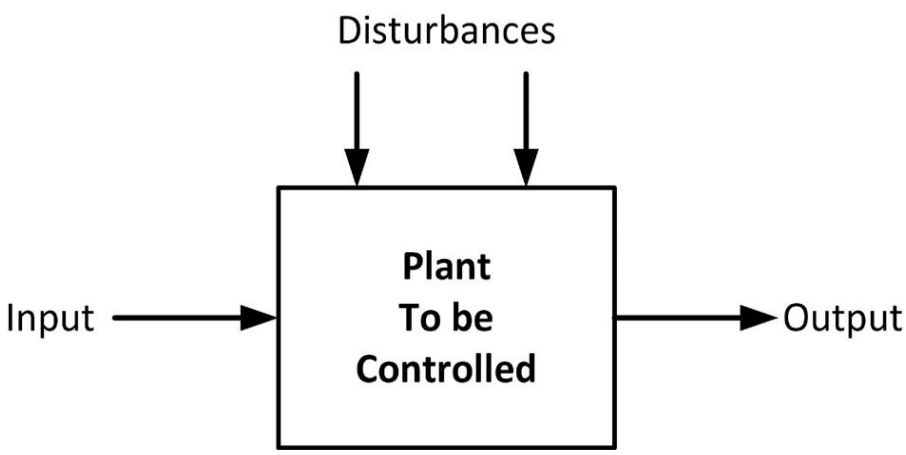
D. Control

1. Introduction

“How do we make the car follow the planned path accurately?”

Self-driving automobiles, sometimes referred to as autonomous vehicles, are a cutting-edge technological advancement that could revolutionize our daily lives and modes of transportation. These cars mimic a human driver’s decision-making process by utilizing a combination of sensors, software, and control systems to make decisions in real time. They are designed to function alone and navigate.

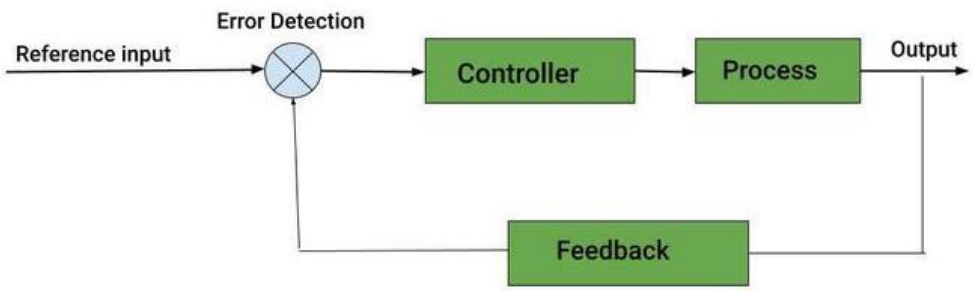
Control theory is the study of how to apply force to an object to control its movement. For self-driving cars, we can apply forces to the vehicle via three "control inputs": steering, throttle, and brake.
The algorithms that determine how to use these control inputs are called "controllers". There are many, many types of controllers, although in the vehicle domain only a few are common and the rest are more specialized. **See references for more control algorithms.**



- The control system receives a desired value, such as the target speed or steering angle, and also measures the vehicle’s actual speed or steering using sensors.
- It compares the desired value with the actual value to calculate the error between them.
- A controller processes this error and decides how much steering, throttle, or braking action is required.
- These control commands are sent to the vehicle, and the car moves according to physical effects like tire grip and load transfer.
- Sensors measure the new vehicle state and send feedback to the controller, and this process repeats continuously to keep the vehicle stable and accurate.

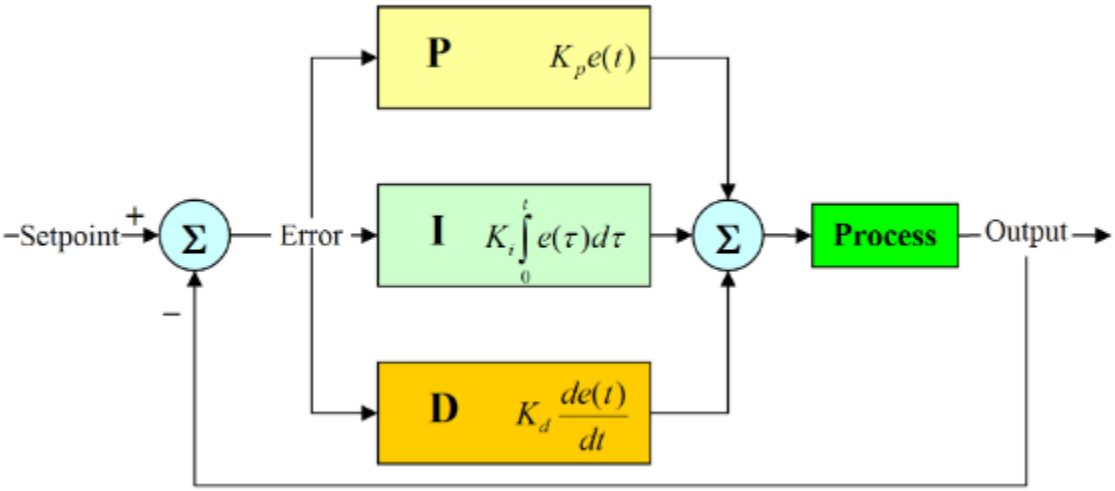
2. Open Loop v/s Closed Loop

- The above image was an example of an open-loop control system.
- The opposite of an open-loop configuration is a closed-loop system. This involves taking feedback from the plant about its current state (via sensors), hence it is “closed”.



- While open-loop systems are easier to set up and implement, closed-loop systems provide better tracking, especially with the presence of external disturbances, non-linearities, etc.

3. PID :



- PID is one of the most commonly used and robust controllers out there. It stands for Proportional-Integral-Derivative Controller. We have 3 constants here : namely K_p, K_i, K_d, each controlling the proportional, integral, and derivative actions respectively.
- PID control is a feedback control method used to make a system reach and maintain a desired value, such as speed or steering angle.
- The controller first calculates the error by comparing the desired value with the actual value measured by sensors.
- The **Proportional (P)** part reacts to the current error and applies a correction that is directly proportional to how big the error is.
- The **Integral (I)** part looks at the accumulated error over time and helps remove any steady-state error that remains after proportional control.
- The **Derivative (D)** part predicts future errors by observing how fast the error is changing and helps reduce overshoot and oscillations.
- The combined P, I, and D actions generate a control output that adjusts steering, throttle, or braking.
- The controller continuously repeats this process using feedback, so the system remains stable and accurate.

Resources for Further Study :

- https://youtu.be/fv6dLTEvl74?si=K6g-O1ziarYJjc5w
- The PID Controller & Theory Explained - NI
- https://www.youtube.com/watch?v=IBC1nEq0_nk
- https://www.youtube.com/watch?v=qYR7mmcwT2w

E. Vehicle Dynamics

1. Load Transfer

Almost everyone who has been in a car or has seen one being driven must have observed that whenever the car undergoes acceleration, the car, along with its passengers, moves in a direction opposite to the car's acceleration. This phenomenon is explained by the basic concept of load/weight transfer. Load transfer that occurs while turning is termed Lateral Load Transfer, and while accelerating or braking, it is termed Longitudinal Load Transfer.

The formula to calculate longitudinal load transfer is,

Here;

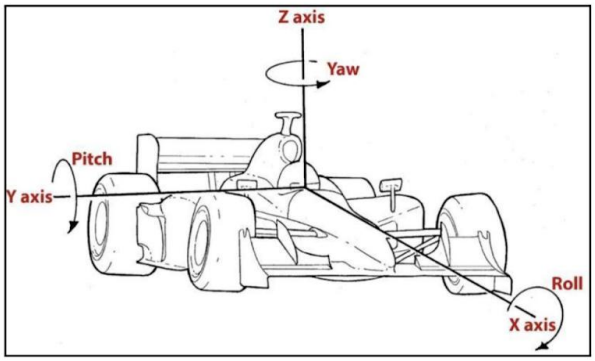
$$\Delta W_X = \frac{h}{\ell} W A_X$$

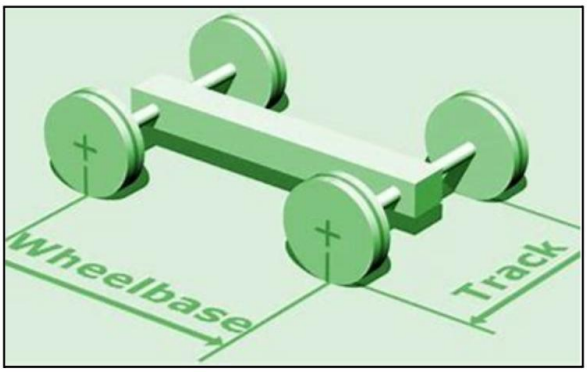
- ΔWx = Increase in the rear axle downward load (or corresponding decrease in front axle downward load)
- h = Height of the center of gravity
- l = Wheelbase of the vehicle
- W = Weight of the vehicle (Mass * Gravity)
- Ax = Longitudinal acceleration

2. Coordinates

Vehicle Axis System:- A reference axis system defined for the entire vehicle. The SAE Axis System is the most commonly used one. Imagine three lines running through a car and intersecting at right angles at the car's centre of gravity. Rotation around the front-to-back axis is called roll. Rotation around the side-to-side axis is called pitch. Rotation around the vertical axis is called yaw.

Basic Dimensions:- The Wheelbase is the distance between the front and rear axles of a vehicle, and the Track Width is the distance between the centerline of each of the two wheels on the same axle on any given vehicle.





3. Steering

I. Role of Steering System

- The steering effort required by the driver should not be too much to make them uncomfortable.
- The steering system needs to be such that the car can maneuver all turns on the track.

II. Types of Steering Systems

Parallel - Geometry -

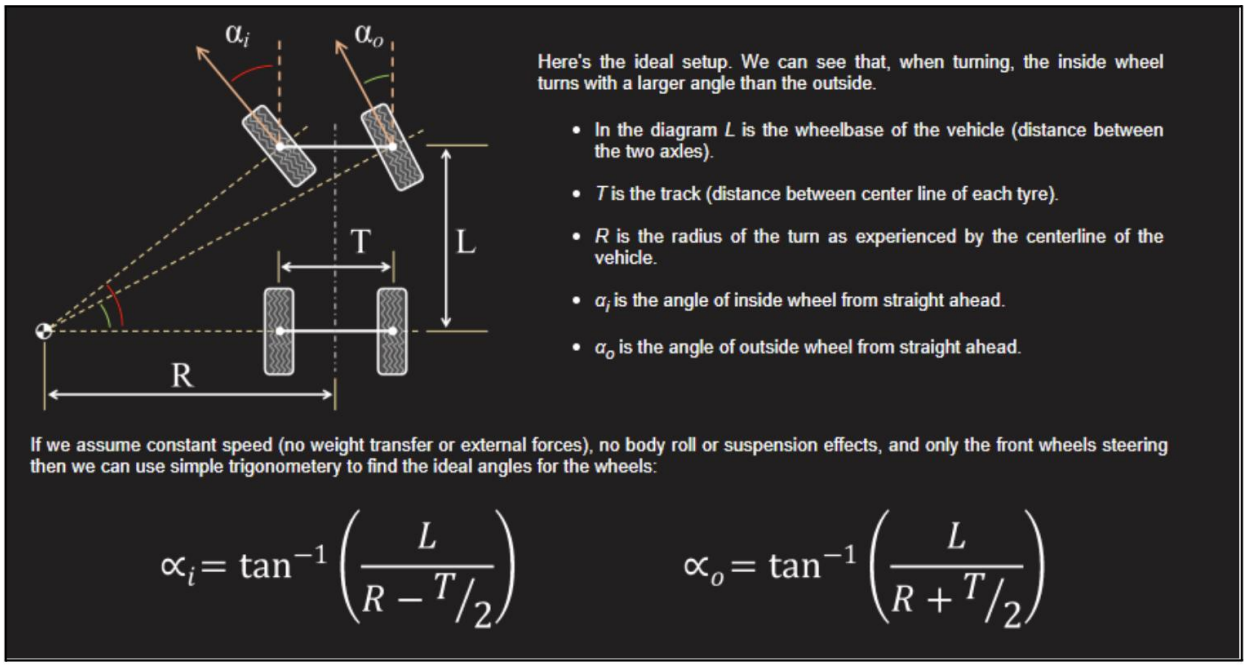
The simplest steering system that can be designed is Parallel steering, which suggests that both the front wheels will turn at an identical angle.

Ackermann - Geometry -

The typical steering system in a road car is the Ackermann steering system. It is governed by the Ackermann principle, which prevents the tyres from slipping sideways while turning. To turn about an instantaneous centre, the inner wheel angle is greater than the outer wheel angle to turn about an instantaneous centre.

Anti-Ackerman Geometry -

Anti Ackermann geometry is the reverse of Ackermann geometry. The outer wheel angle is greater than the inner wheel angle. This type of geometry is preferred by most racing cars.



4. Braking

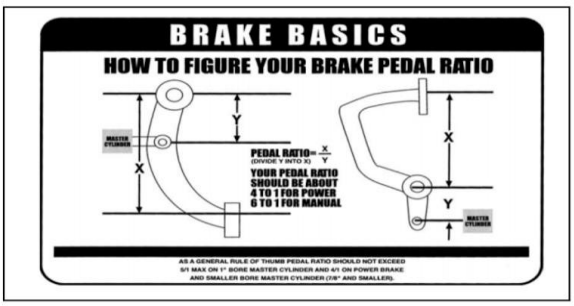
Braking is the process of slowing down or stopping a vehicle. A hydraulic braking system (like the one used in most cars with disc brakes) works as follows:

- Pressing the Brake Pedal
  - When you press the brake pedal, you apply a force.
  - This force is amplified by the pedal ratio (P) to make braking easier.
- Hydraulic System Activation
  - The force is transmitted to the master cylinder, which contains brake fluid.
  - This fluid gets compressed and travels through hydraulic lines to the brake calipers.
- Brake Caliper and Disc Interaction
  - The brake calipers have pistons that push brake pads against the rotating brake disc (rotor).
  - This creates friction, which slows down the wheel.
- Stopping the Vehicle
  - The friction between the brake pads and disc produces braking torque (opposite to wheel rotation).
  - This torque helps bring the vehicle to a stop.
- Releasing the Brakes
  - When you release the brake pedal, the pressure in the hydraulic system decreases.
  - The pistons move back, and the brake pads stop pressing the disc, allowing the wheels to rotate freely.

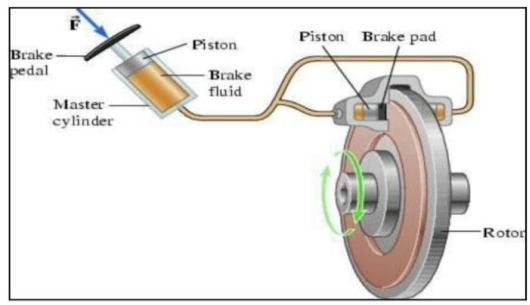
Key Concepts in Braking Calculations

- Pedal Force (Fp): Force applied on the brake pedal.
- Pedal Ratio (P): Helps amplify the pedal force.
- Hydraulic Gain (Ac/Ap): Further amplifies the force using brake fluid pressure.
- Braking Torque (Tb): The force applied on the brake disc to stop rotation.
- Wheel Radius (Rw): Determines how braking force is distributed.

To simplify the calculation, we can reduce the four wheels into a single (equivalent system) wheel to simplify the calculation. Ignoring all the other resistive forces (like rolling resistance, aerodynamic drag,etc.), we can do the preliminary braking calculations.



Fig(9): Pedal Ratio



Fig(10): Braking system

Let the vehicle's mass be M, acceleration (retardation) be a, and R_w is the radius of the wheel/tyre. F_b is the equivalent braking force at all the wheels, and F_b is the force applied at the brake pedal.

$$F_b = M \times a = \frac{T_b}{R_w} \qquad T_b = 2 \times \left(R_b \times \left(F_P \times P \times \frac{A_c}{A_p}\right)\right)$$

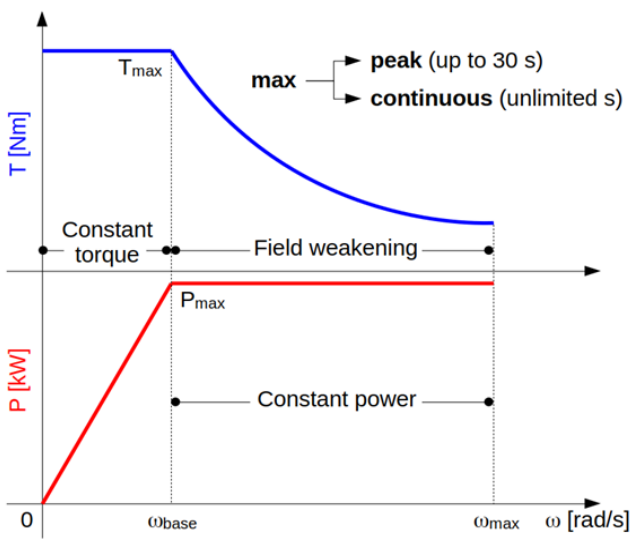
Using these equations, we can get the relation b/w a (acceleration) and the F_p (Pedal Force):

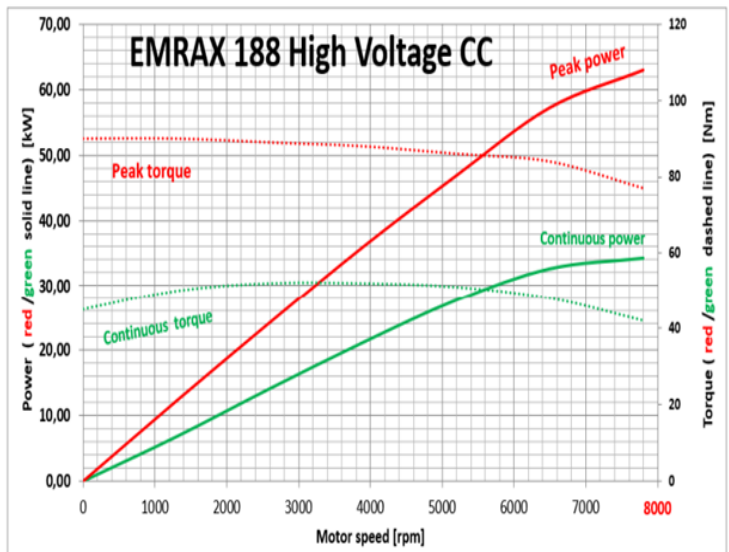
$$F_P = \frac{(M \times a \times R_w)}{2 \times \left(R_b \times P \times \frac{A_c}{A_p}\right)}$$

F. Powertrain

1. Torque-RPM Curve

I. Initial Region (Constant Torque)





- At low RPM, the motor provides constant torque because the current supply is regulated to deliver the maximum possible torque.
- Power is defined as :

$$P = T \cdot \omega$$

(where T is torque and ω is angular velocity in radians per second).

- As ω increases, the power also increases linearly from zero to its maximum value, Pmax.

II. Transition to Maximum Power (Base Speed)

- The point where the power reaches Pmax is called the base speed.
- Beyond this speed, maintaining constant torque would require more power than the motor can provide, so torque starts to decrease.

III. Constant Power Region (Hyperbolic Torque Decrease)

- In this region, the motor operates at constant power, P=Pmax.

- To maintain Pmax as ω continues to increase, the torque T must decrease according to the relationship:

$$T = \frac{P_{max}}{\omega}$$

- This decrease follows a hyperbolic curve because torque is inversely proportional to angular velocity.

2. Powertrain Configurations

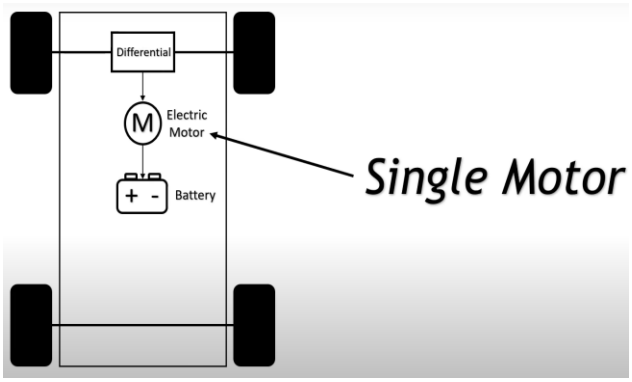
Electric vehicles (EVs) use different motor configurations to power their wheels. Here's a simplified explanation of three common setups:

I. Single Motor Configuration

Only one motor powers either the front or rear axle (axle: shaft connecting to both left and right wheels); and uses a differential to distribute power and torque between the left and right wheel. This setup is simple and cost-effective, ideal for everyday driving. However, it offers less control over traction and cornering.

- Simpler control algorithms focus on acceleration and braking.
- Limited capability for advanced traction control.

Differential: A differential allows wheels on the same axle to rotate at different speeds, improving cornering by compensating for the varying distances each wheel travels during turns. Nowadays, there are differentials that can be electronically and algorithmically controlled for torque/power distribution.



II. Dual Motor Configuration

Two independent motors, one for each wheel (left and right) on the same axle (either rear or front). This setup provides better control and allows for torque vectoring on that axle.

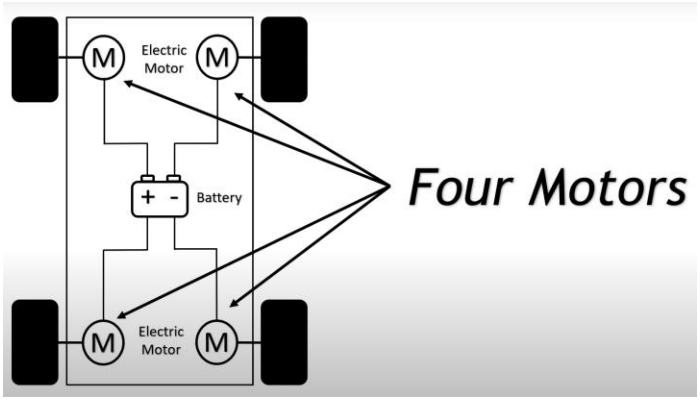
- Improved traction during acceleration and enhanced handling.
- Algorithms focus on distributing power/torque between left and right motors.
- Better wheel slip detection and control for improved performance on slippery surfaces.

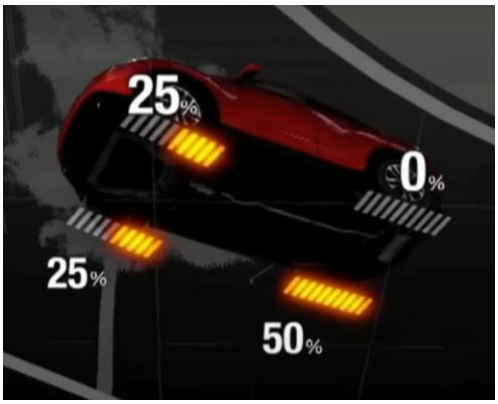
Torque vectoring: It dynamically adjusts power distribution between wheels, enhancing traction, stability, and cornering in cars. For algorithm development in autonomous vehicles, it requires optimization of wheel-specific torque based on real-time inputs from sensors.

III. Four Motor Configuration

One motor for each wheel, providing independent control of all four wheels. This setup offers the best traction, cornering, and off-road performance.

- Ultimate precision in torque distribution, torque vectoring, performance and stability.





Besides, one can see how all 4 wheels' motors have different torque distribution, while cornering the car.

3. Motor and its Types

Motors in electric vehicles convert electrical energy from the battery into rotational mechanical energy. This energy is then transmitted through the drivetrain, ultimately causing the wheels to rotate and propel the vehicle forward. Unlike IC engines, electric motors provide very high efficiency in a broad range of speed and torque, so most EVs don't have multiple gear drives. Many kinds of motors, like BLDC, PMSM, and Induction motors, are being used in various segments of EVs. PMSM motors generate more torque and have higher efficiency, so they are used in most electric 4-wheelers nowadays.

Resource Link for drivetrain configurations and types of motors : [Types of Motors used in EV | Single, Dual, Three & Four Motor Configuration in EV]

- **Control:** All these motors typically use PWM signals and closed-loop feedback, often employing Field-Oriented Control (FOC) or Direct Torque Control (DTC) techniques.

I. Motor Control Unit

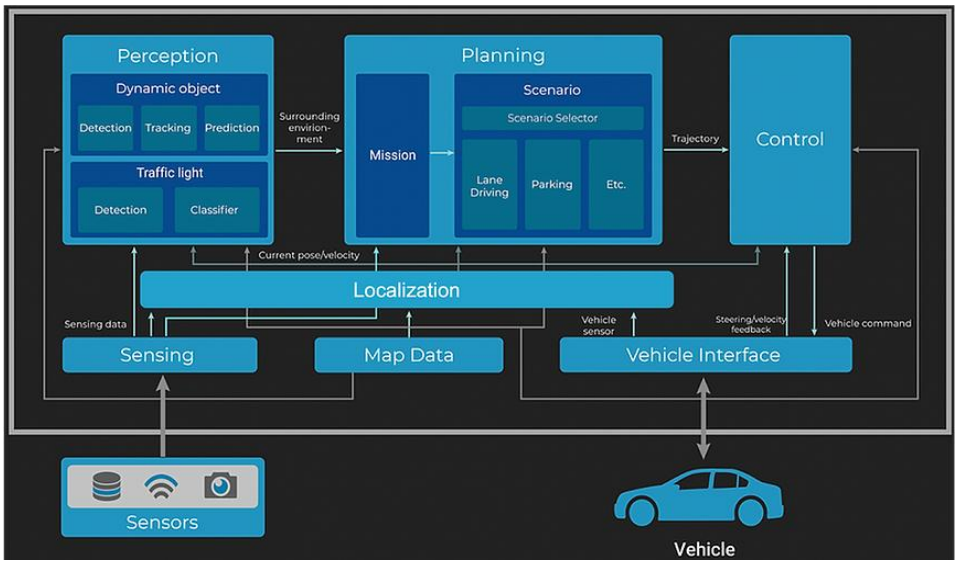
- Throttle input in vehicles translates to motor commands by interpreting the driver's acceleration demand. Sensors capture the throttle position, and the Electronic Control Unit (ECU) processes the signal. It varies the current/voltage/power output to change motor's speed and torque based on throttle input.
- Further, the Motor Control Unit (MCU) transforms the battery's direct current into alternating current using a Voltage Source Inverter (VSI) by applying pulse width modulated (PWM) waveforms to the gates of VSI switches/MOSFETs. PWM pulses are generated according to the position sensors (Hall/Encoder/Resolver) feedback and the ON Time/Duty Cycle of these pulses varies according to the required Speed/Torque command. To achieve efficient closed loop control of Motor drive, Vector Control methods/Field Oriented Control (FOC) are adopted.

G. Robot Operating System (ROS)

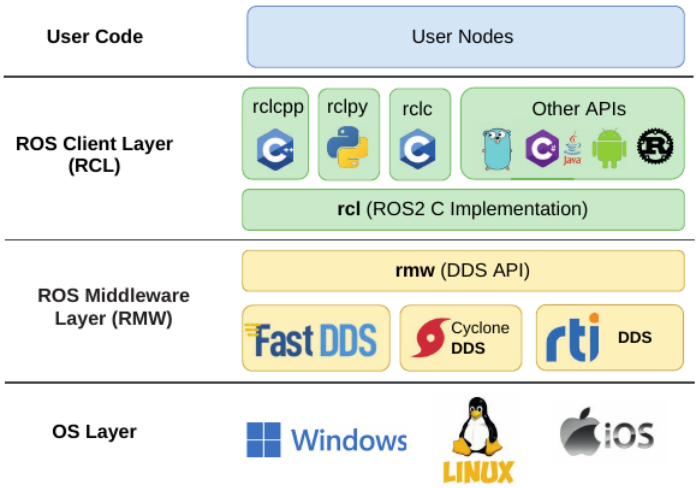
1. ROS and its use in Autonomous Vehicles:

ROS is a flexible framework that helps developers build robot applications. It is not actually an operating system, rather a collection of tools, libraries, and conventions that simplify the complex task of creating robust robot behaviors.

Autonomous vehicles need to handle multiple tasks simultaneously - they need to understand their environment through sensors, plan their path, control their movement, and make split-second decisions. ROS provides the infrastructure to manage all these tasks efficiently. For example, one component might process camera data to detect obstacles, while another plans the safest route, and yet another controls the steering and acceleration.



ROS2 is particularly well-suited for autonomous vehicles because it can handle the demanding requirements of real-time processing and safety-critical operations. ROS2 has a highly sophisticated modular software architecture where different functionalities (e.g., sensor fusion, decision-making, and motion control) interact smoothly



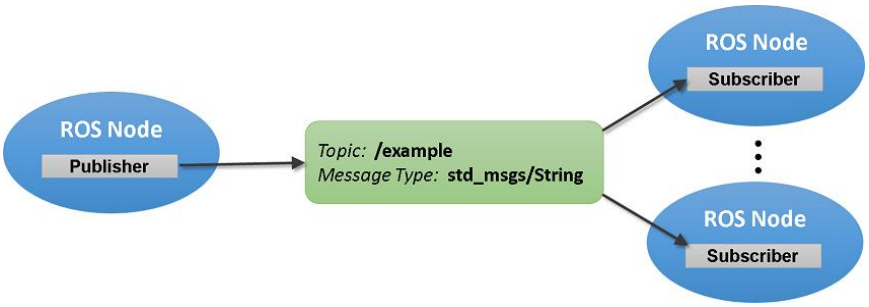
Now, let's look at the core components of ROS2:

ROS2 follows a distributed computing model, where different components run as independent nodes. These nodes communicate through:

- Topics → For continuous data streaming (e.g., LiDAR data).
- Services → For request-response operations.
- Actions → For long-running tasks like motion planning.

A term you will read a lot is the Publisher-Subscriber Mode

Message passing in ROS happens with the Publisher Subscriber Interface provided by ROS library functions.



A ROS Node can be a Publisher or a Subscriber. A Publisher is the one that puts the messages of some standard Message Type to a particular Topic. The Subscriber on the other hand subscribes to the Topic so that it receives the messages whenever any message is published to the Topic.

Also, publishers and subscribers are not aware of each others' existence. The idea is to decouple the production of information from its consumption and all the IP addresses of various nodes are tracked by the ROS Master.

For SLAM (Simultaneous Localization and Mapping), different nodes handle sensor data processing, map building, and position estimation. They communicate through topics to share sensor data and map updates.

In perception, nodes process data from various sensors (cameras, lidar, radar) and publish their findings on topics. Other nodes subscribe to this information to build a comprehensive understanding of the environment.

Path planning uses this perceived information, subscribing to relevant topics to understand the environment and obstacles. It then publishes planned paths that the control nodes can follow.

2. Use of Version Control in ROS:

ROS exists to aid integration of the three major components within the entire pipeline. However, to effectively communicate and develop upon different ideas, it is important that the codebase is maintained for easy access and readability. Thus, Version Control via GIT becomes a key component when operating on various tasks.
You are free to look at some common use git commands allowing you to access and contribute to codebases.

Access the cheat sheet from here: https://education.github.com/git-cheat-sheet-education.pdf

3. Linux, Virtual Machines & Programming Languages:

ROS is best operated on a Linux based system, to efficiently run simulations on platforms like Rviz2 or Gazebo.

Virtual Machines allow the use of Linux as a platform on an existing OS without having the need to dual boot the machine. However, the leads to a lack of allocation of resources as opposed to a singly run Operating System. Hence, for efficiency, the code base must be altered to be able to run them on the available system configurations.

Robotic programming largely involves python and C++ as their major engines. Thus, idea of at least one of the languages is important to properly work with system commands and ROS based simulations. Although we do not expect you to be proficient in these programming languages before being recruited, a basic idea of programming and use is much appreciated.

You may also look at some basic commands to run on your Linux from here if you're new to it:
https://www.geeksforgeeks.org/linux-unix/linux-commands-cheat-sheet/

REFERENCES:

https://docs.ros.org/en/rolling/Tutorials/Beginner-CLI-Tools.html

https://docs.ros.org/en/rolling/Concepts/Basic.html

https://medium.com/swlh/part-3-create-your-first-ros-publisher-and-subscriber-nodes-2e833dea7598

https://mpolinowski.github.io/docs/Automation_and_Robotics/Robotics_Simulation/2023-11-24--ros2-graph/2023-11-24/

ALL THE BEST!